

Assignment 4

Kumar Kanishk Singh

2024-03-28

Question 1

We fit the given model using JAGS.

```
n<-6
Y<-c(64, 13, 33, 18, 30, 20)
t <- 1:n

alpha<-dnorm(0,10000)
beta<-dnorm(0,10000)
lambda<-exp(alpha+beta*t)
data<-list(Y=Y,n=n)

library(rjags)

## Warning: package 'rjags' was built under R version 4.2.3
## Loading required package: coda
## Warning: package 'coda' was built under R version 4.2.3
## Linked to JAGS 4.3.1
## Loaded modules: basemod,bugs

model_string<-textConnection("model{

  #Likelihood
  for(t in 1:n){
    Y[t]~dpois(lambda[t])
  }

  #Priors
  alpha~dnorm(0,10000)
  beta~dnorm(0,10000)
  for(t in 1:n){
    lambda[t]=exp(alpha+beta*t)
  }
}")

#inits <- list(alpha=alpha, beta=beta)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

update(model, 2000, progress.bar = "none")

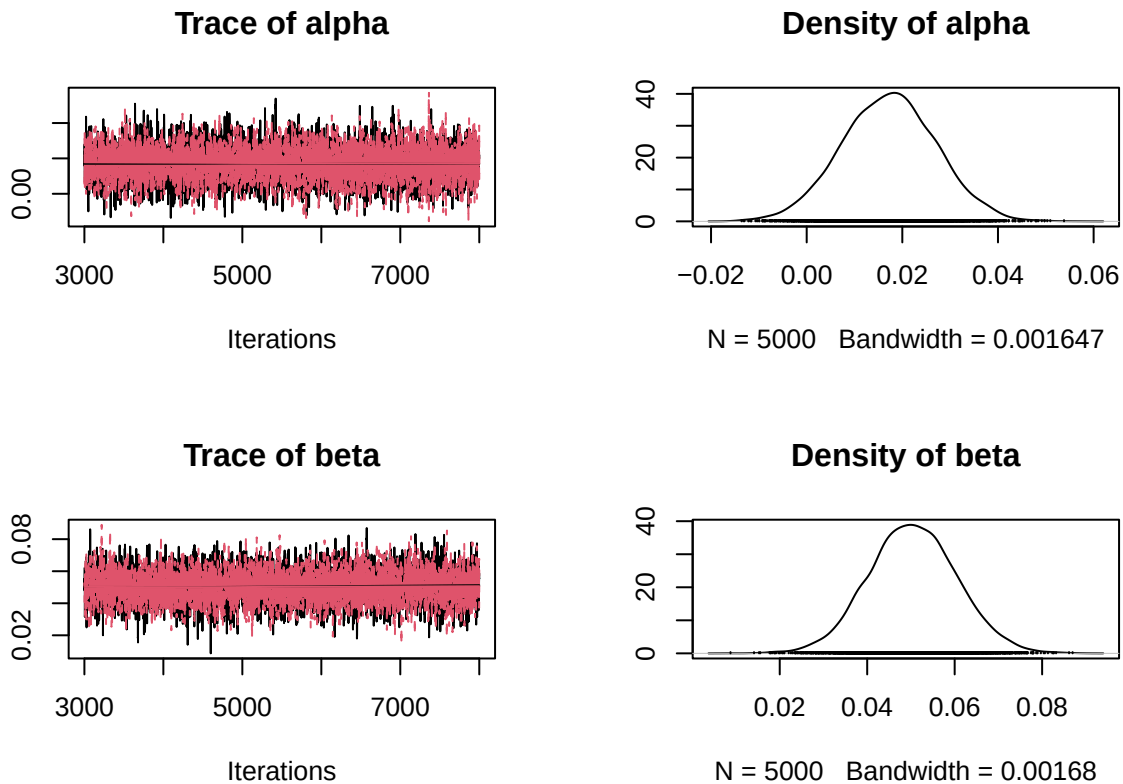
params <-c("alpha","beta")
```

```

samples <- coda.samples(model,
  variable.names = params,
  n.iter = 5000, progress.bar = "none")

plot(samples)

```



```
summary(samples)
```

```

##
## Iterations = 3001:8000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##      Mean      SD Naive SE Time-series SE
## alpha 0.01721 0.009806 9.806e-05 0.0001229
## beta  0.05041 0.010035 1.004e-04 0.0001261
##
## 2. Quantiles for each variable:
##
##      2.5%    25%    50%    75%    97.5%
## alpha -0.00212 0.01054 0.01730 0.02383 0.03659
## beta   0.03097 0.04365 0.05036 0.05705 0.07038

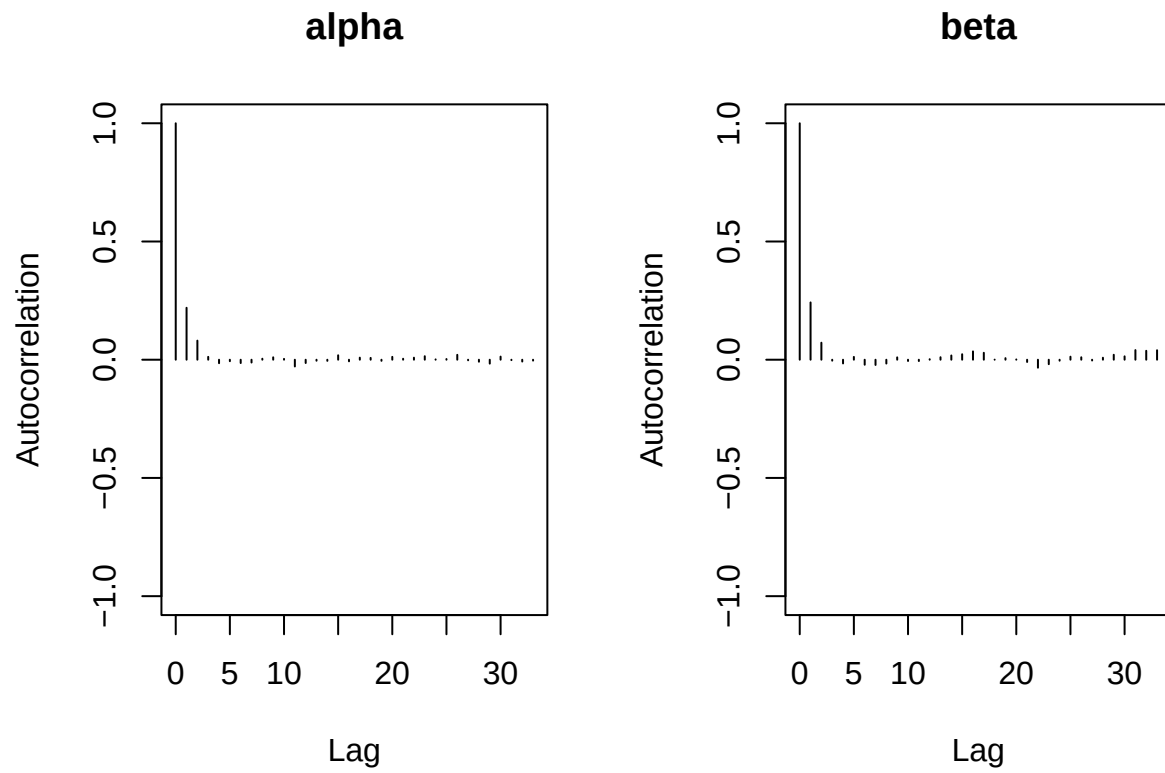
```

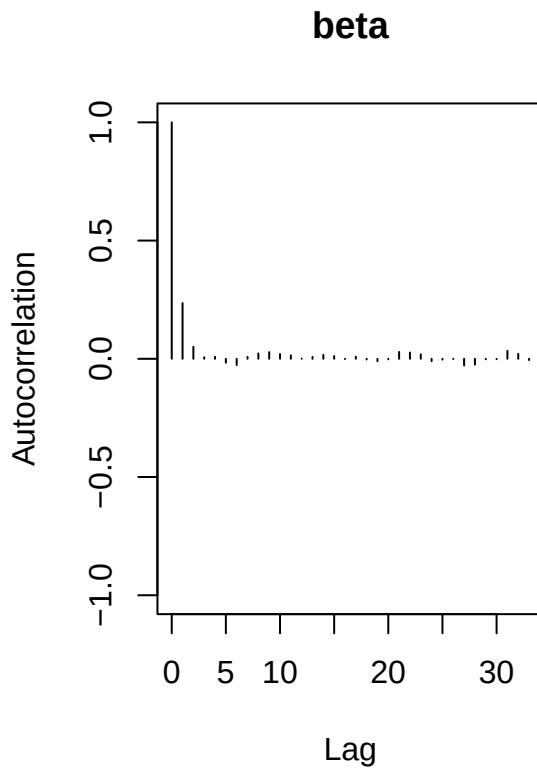
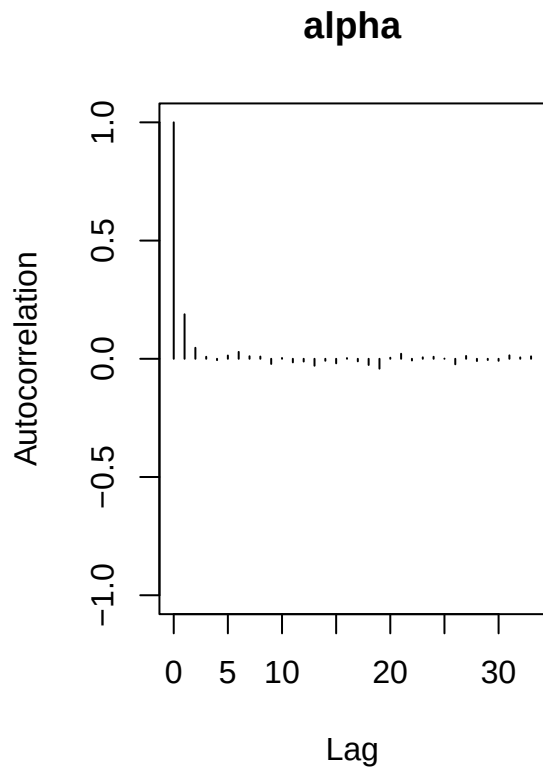
The trace plot indicates convergence. Now let's look at autocorrelation plots. The autocorrelation is low after lag 2. This also indicates convergence.

```
library(coda)
```

```
# Low autocorrelation indicates convergence
```

```
autocorr.plot(samples)
```





```
autocorr(samples, lag = 1)
```

```
## [[1]]
## , , alpha
##
##          alpha          beta
## Lag 1 0.2196772 0.01677296
##
## , , beta
##
##          alpha          beta
## Lag 1 -0.03705444 0.2423725
##
##
## [[2]]
## , , alpha
##
##          alpha          beta
## Lag 1 0.1881819 0.001785828
##
## , , beta
##
##          alpha          beta
## Lag 1 0.01907434 0.2358491
```

High ESS indicates convergence

```
effectiveSize(samples)
```

```
##      alpha      beta  
## 6402.025 6332.737
```

R less than 1.1 indicates convergence

```
gelman.diag(samples)
```

```
## Potential scale reduction factors:
```

```
##
```

```
##      Point est. Upper C.I.
```

```
## alpha          1          1
```

```
## beta           1          1
```

```
##
```

```
## Multivariate psrf
```

```
##
```

```
## 1
```

/z/ less than 2 indicates convergence {r} `geweke.diag(samples[[1]])` We can also see that the rate of discovery increases with time.

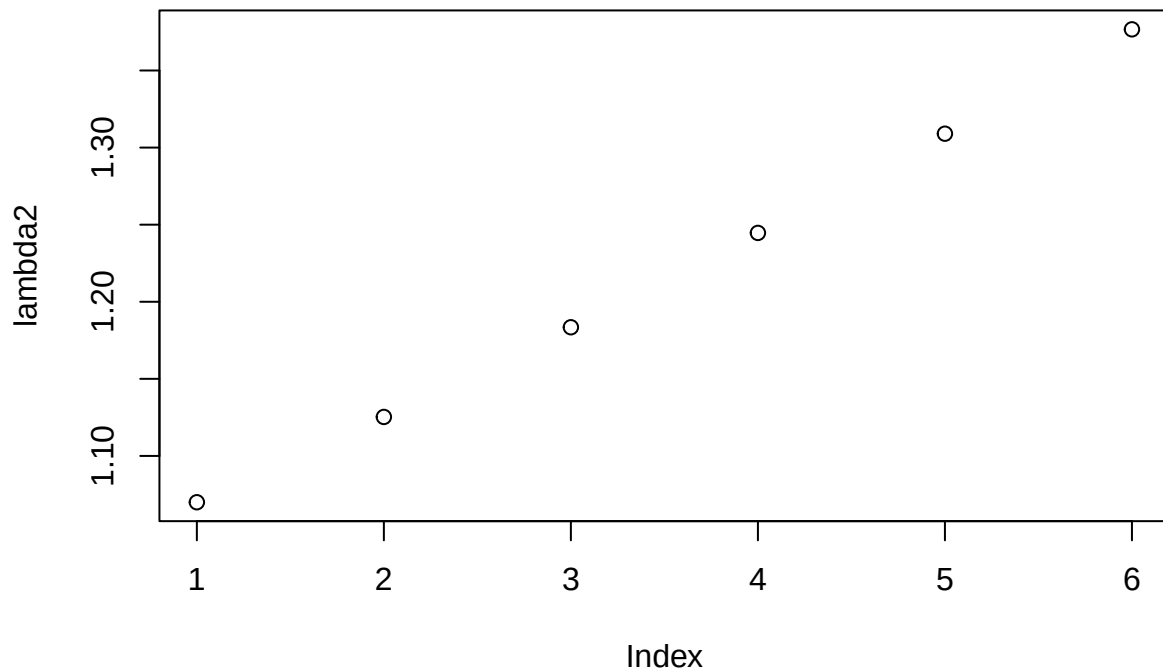
```
t<-1:n
```

```
a<-summary(samples)$statistics[1,1]
```

```
b<-summary(samples)$statistics[2,1]
```

```
lambda2<-exp(a+b*t)
```

```
plot(lambda2)
```



Lets set up a metropolis sampler for this problem.

```
n<-6
Y<-c(64, 13, 33, 18, 30, 20)
t <- 1:n

beta<-c(3,0)

iter<-25000
par<-matrix(0,iter,2)

cand<-c(0.4,0.06)

posterior<-function(Y,t,beta,sd=10){
  l<-exp(beta[1]+t*beta[2])
  lambda<-prod(dpois(Y,l))
  p<-prod(dnorm(beta,0,sd))
  return(lambda*p)}
#can<-rnorm(1,beta[1],cand[1])
#posterior(Y,t,can)

for(i in 1:iter)
{
  for(j in 1:2)
  {
    can<-beta
    can[j]<-rnorm(1,beta[j],cand[j])
```

```

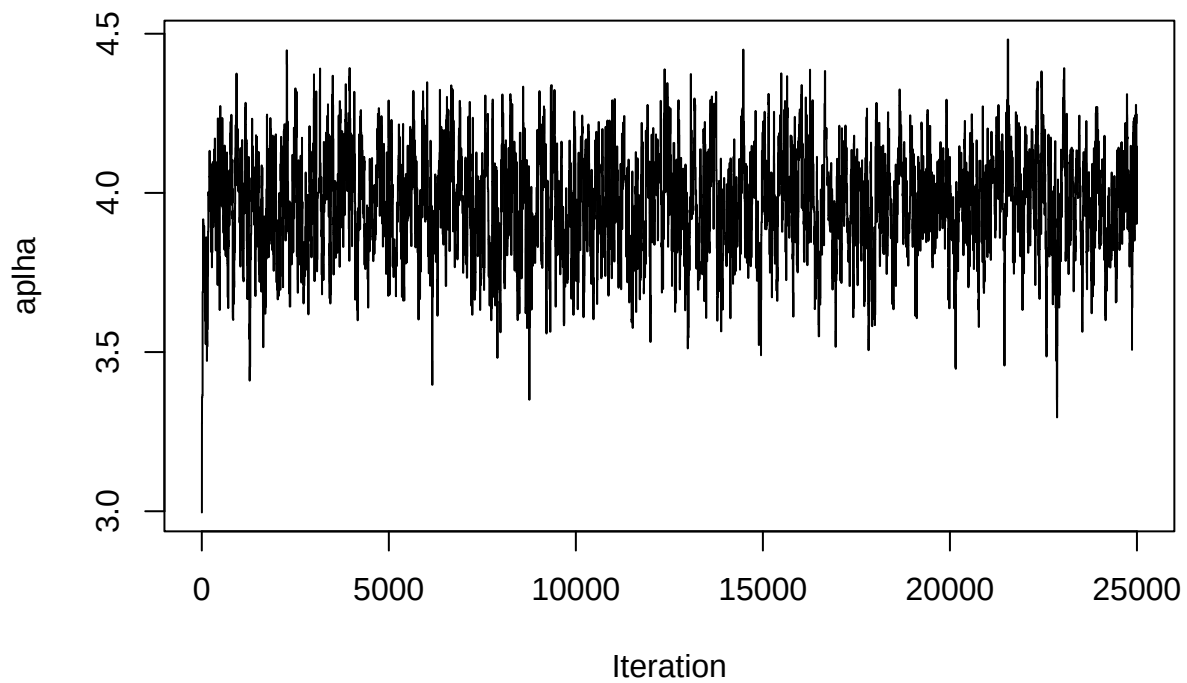
R<-posterior(Y,t,can)/posterior(Y,t,beta)

if(runif(1)<R)
{
  beta<-can
}

par[i,]<-beta
}

plot(par[,1],type="l",ylab=expression(alpha),xlab="Iteration")

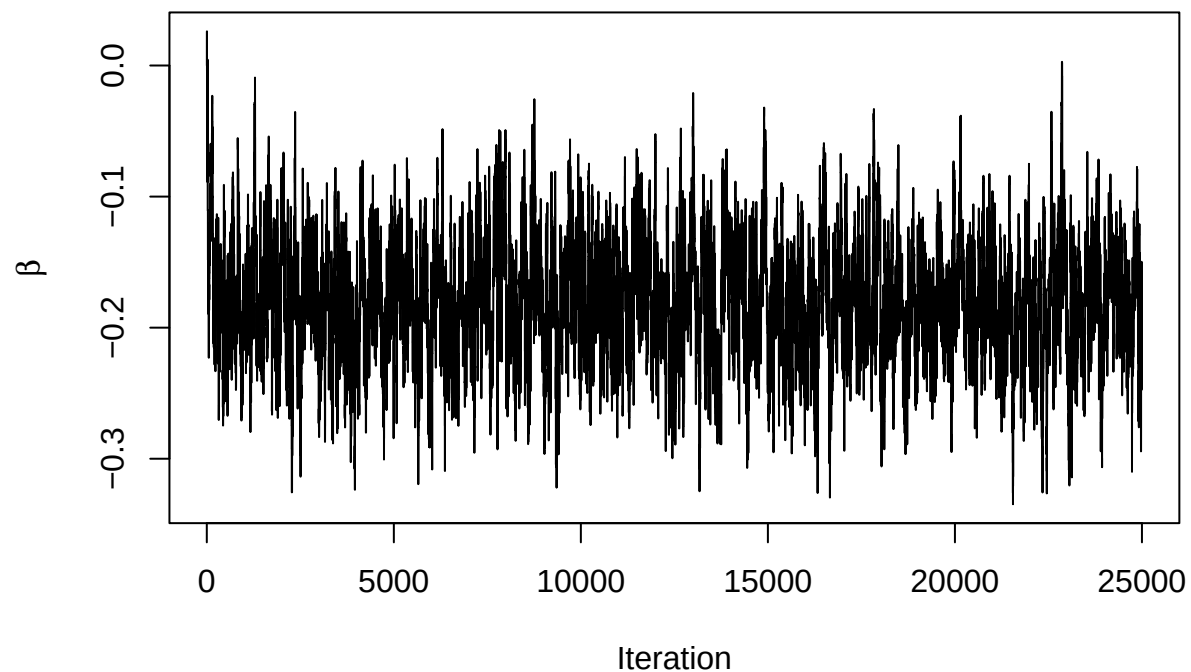
```



```

plot(par[,2],type="l",ylab=expression(beta),xlab="Iteration")

```



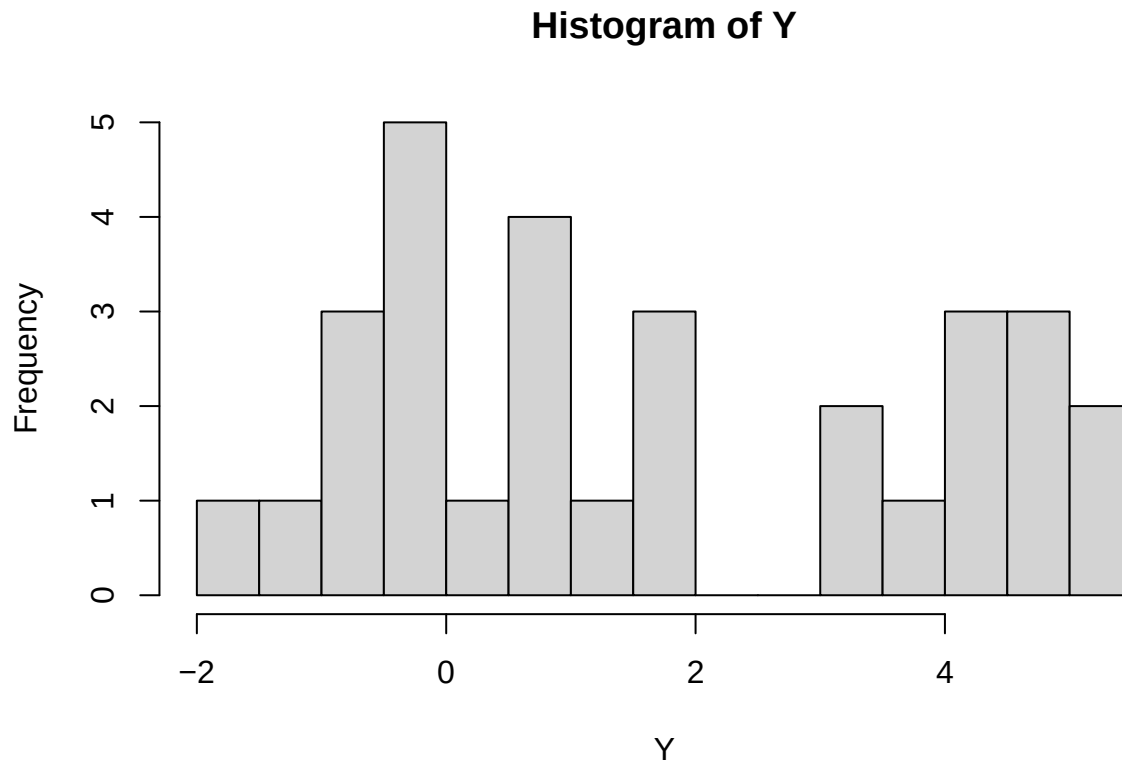
The trace plots indicate that the model has converged. Following are the acceptance ratios:

```
acc_rate <- colMeans(par[-1,] != par[-iter,])
acc_rate
```

```
## [1] 0.2283291 0.4048562
```

Question 2

```
set.seed(27695)
theta_true <- 4
n          <- 30
B          <- rbinom(n,1,0.5)
Y          <- rnorm(n,B*theta_true,1)
hist(Y,breaks=25)
```

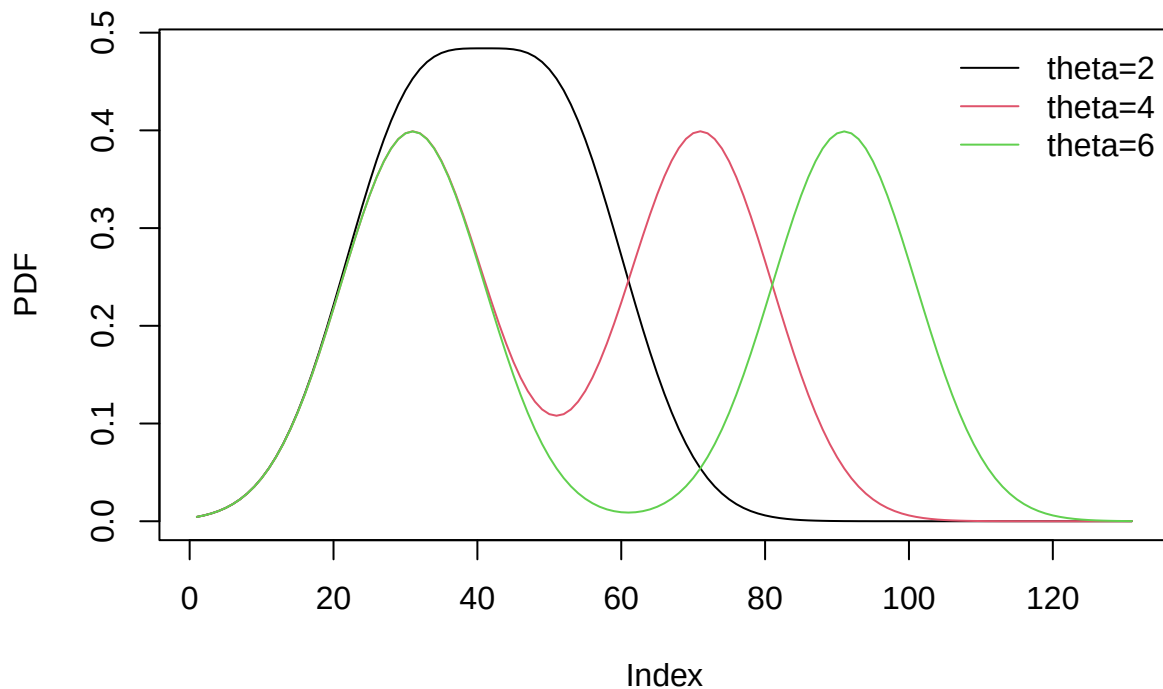



b. Following are the plots for $f(Y|\theta)$ for $\theta=\{2,4,6\}$:

```
dens<-function(y)
{
  return(exp(-(y^2)/2)/sqrt(2*pi))
}
func<-function(y,theta)
{
  return(dens(y)+dens(y-theta))
}

y<-seq(-3,10,0.1)

plot(func(y,2),type="l",ylab="PDF")
lines(func(y,4),col=2)
lines(func(y,6),col=3)
legend("topright",c("theta=2","theta=4","theta=6"),lty=1,col=1:3,bty="n")
```



c. Following are the MAP estimate of theta and its standard deviation:

```
#install.packages("stats4")
library(stats4)
nlp <- function(theta,Y){
  like<- 0.5*dnorm(Y,0,1) + 0.5*dnorm(Y,theta,1)
  prior<- dnorm(theta,0,10)
  neg_log_post <-sum(log(like))- log(prior)
  return(neg_log_post)}
#map_est <- mle(nlp,start=list(theta=1), fixed=list(Y=Y))
#sd<- sqrt(vcov(map_est))
out <- suppressWarnings(optim(par = 1, nlp, Y = Y, hessian = T))

map_est <- out$par
sd <- sqrt(1 /c(out$hessian))
map_est; sd
```

```
## [1] 4.178906
```

```
## [1] 0.3389554
```

d. Following are the posterior densities of theta:

```
posterior <- function(theta,Y,k){
  post <- dnorm(theta,0,sqrt(10^k))
  for(i in 1:length(Y)){
    post<-post*(0.5*dnorm(Y[i],0,1)+
                0.5*dnorm(Y[i],theta,1))
  }
```

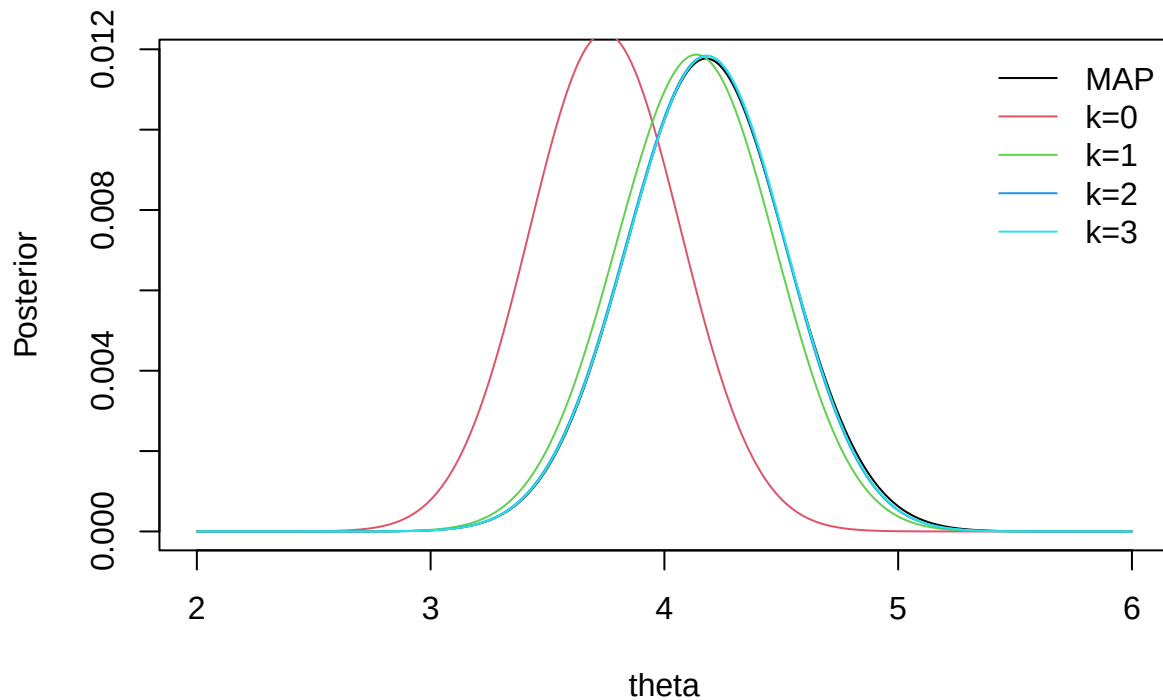
```

    return(post/sum(post))}

theta <- seq(2,6,0.01)
map <- dnorm(theta,map_est,sd)

plot(theta,map/sum(map),col=1,type="l",ylab="Posterior")
lines(theta,posterior(theta,Y,0),col=2)
lines(theta,posterior(theta,Y,1),col=3)
lines(theta,posterior(theta,Y,2),col=4)
lines(theta,posterior(theta,Y,3),col=5)
legend("topright",c("MAP", "k=0", "k=1", "k=2", "k=3"),
      col=1:5,lty=1,bty="n")

```



e.Now we use JAGS to fit this model:

```

data<-list(Y=Y,n=n)

library(rjags)

model_string<-textConnection("model{

  #Likelihood
  for(i in 1:n){
    Y[i]~dnorm(B[i]*theta,1)
  }
}

```

```

#Priors
for(i in 1:n){
  B[i]~dbern(0.5)
}
theta~dnorm(0,1e-2)
})

#inits <- list(alpha=alpha, beta=beta)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

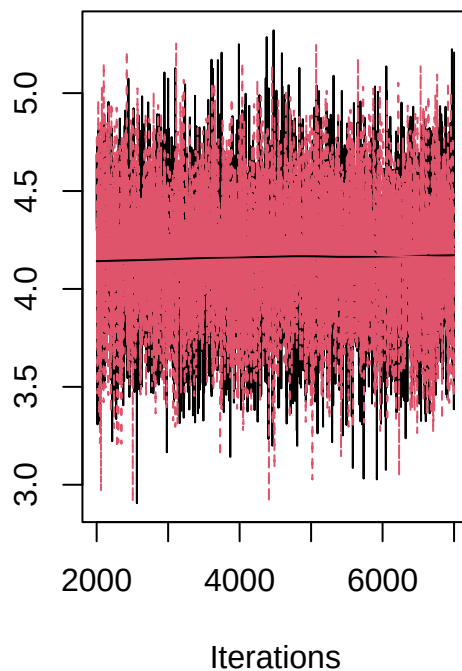
update(model, 2000, progress.bar = "none")

params <-c("theta")
samples <- coda.samples(model,
                        variable.names = params,
                        n.iter = 5000, progress.bar = "none")

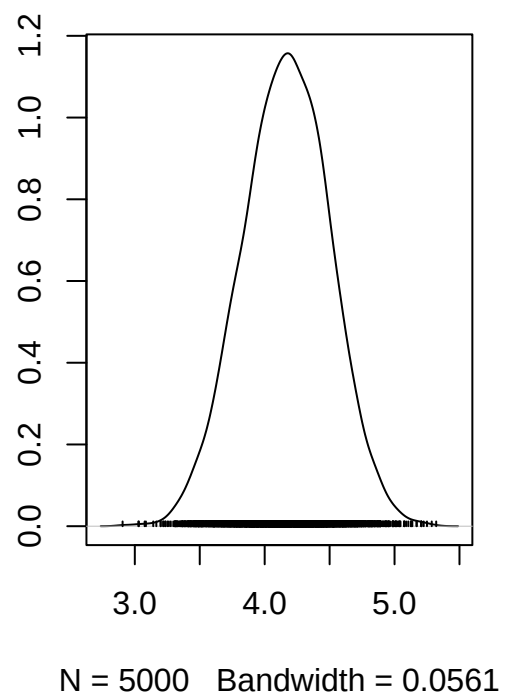
plot(samples)

```

Trace of theta



Density of theta



```
summary(samples)
```

```

##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000

```

```
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean           SD       Naive SE Time-series SE
##      4.164498      0.333945    0.003339      0.004265
##
## 2. Quantiles for each variable:
##
##  2.5%   25%   50%   75%  97.5%
## 3.502 3.938 4.170 4.397 4.813
```

We can see that in both part d and e the mean is around 4 and the height of the curve is slightly less than the 0.012.

Question 3

a. Convergence could be slow because there are more parameters than observations and so it is likely that not all parameters are identifiable.

b. We set up a model using JAGS with $\lambda = 10$ and $a = b = 1$.

```
Y<-10

data<-list(Y=Y)

library(rjags)

model_string<-textConnection("model{

  #Likelihood
  Y~dbin(p,n)

  #Priors
  lambda<-10
  a<-1
  b<-1
  n~dpois(lambda)
  p~dbeta(a,b)
  np<-n*p
}")

#inits <- list(n=20, p=0.5)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

update(model, 2000, progress.bar = "none")

params <- c("n", "p", "np")

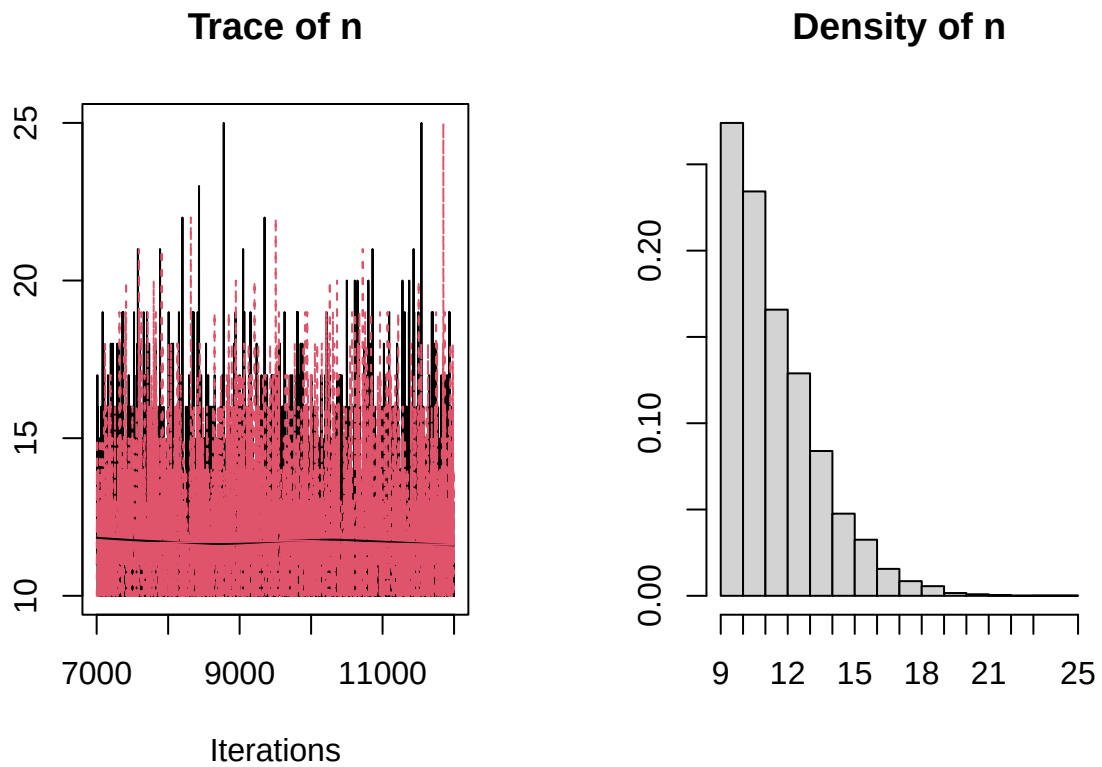
samples <- coda.samples(model,
                        variable.names = params,
                        n.iter = 5000, progress.bar = "none")
samples_n <- coda.samples(model,
                          variable.names = "n",
```

```

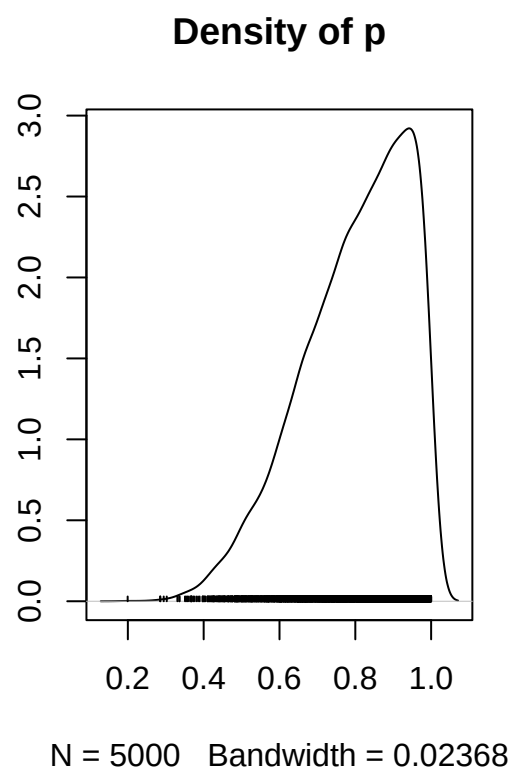
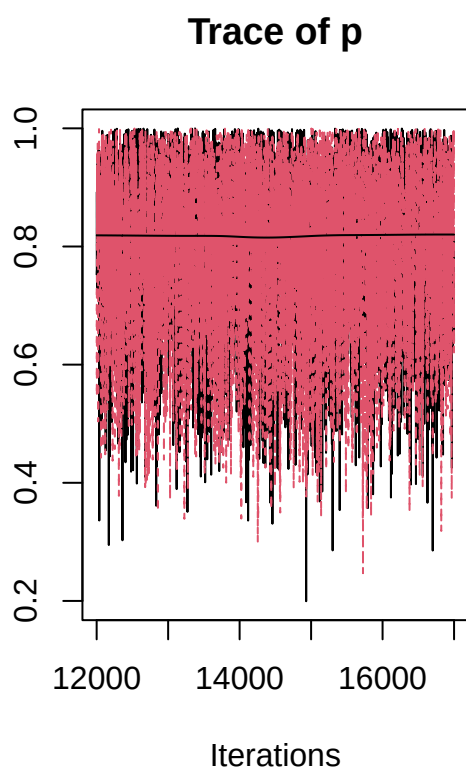
      n.iter = 5000, progress.bar = "none")
samples_p <- coda.samples(model,
      variable.names = "p",
      n.iter = 5000, progress.bar = "none")
samples_np <- coda.samples(model,
      variable.names = "np",
      n.iter = 5000, progress.bar = "none")

plot(samples_n)

```

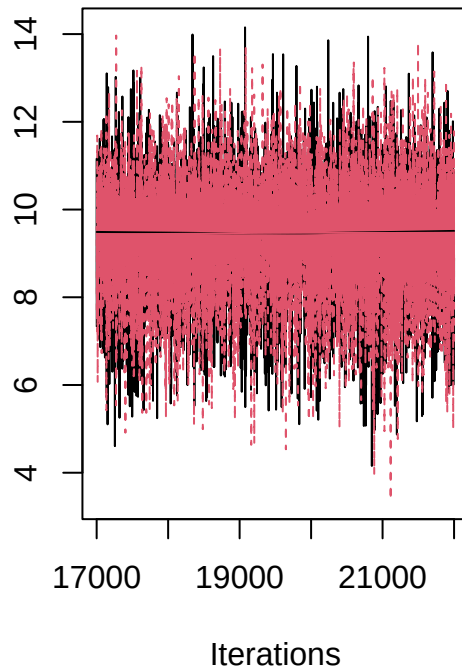


```
plot(samples_p)
```

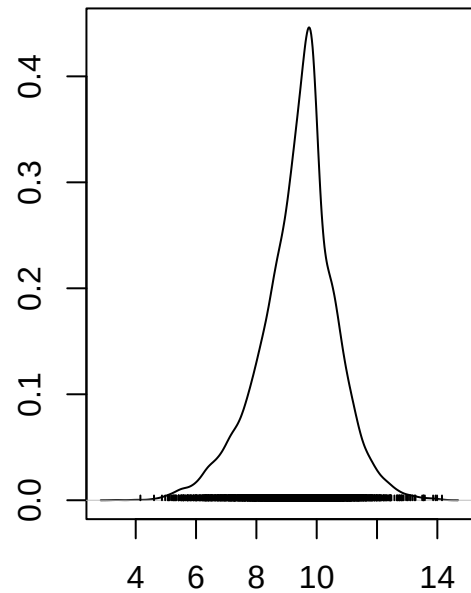


```
plot(samples_np)
```

Trace of np



Density of np



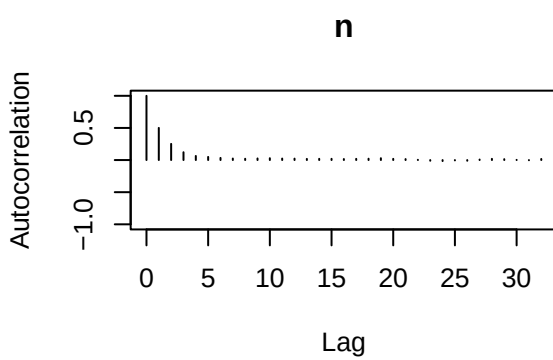
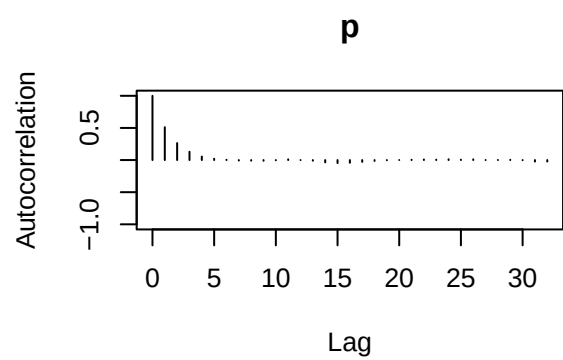
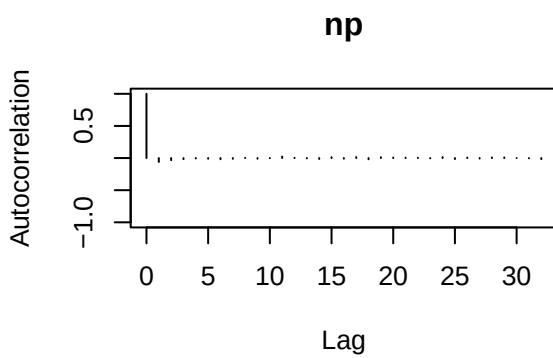
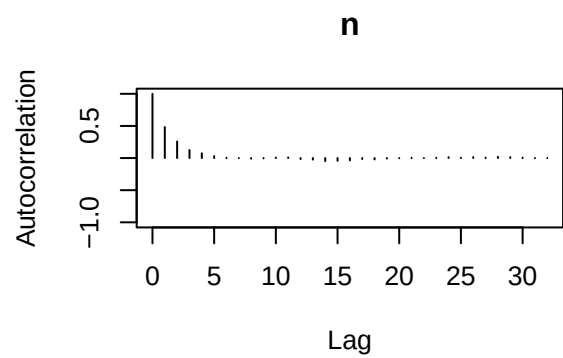
N = 5000 Bandwidth = 0.1776

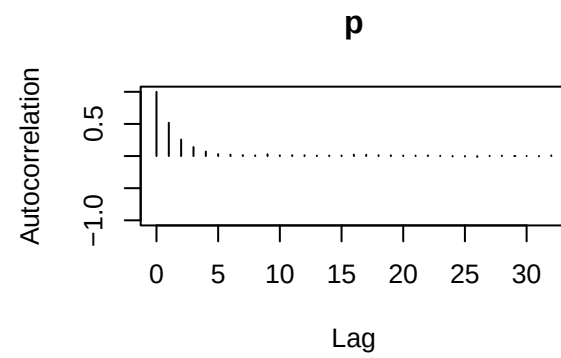
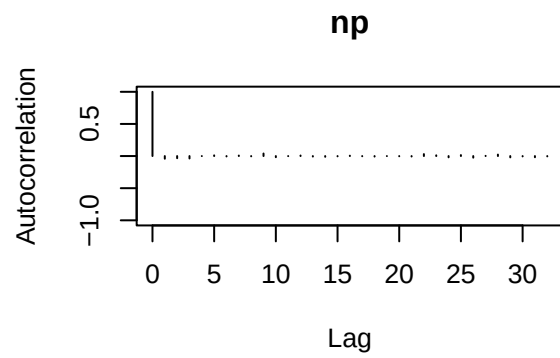
```
summary(samples)
```

```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##      Mean      SD Naive SE Time-series SE
## n  11.9760  1.9778 0.019778      0.034444
## np  9.4036  1.2341 0.012341      0.010966
## p   0.8018  0.1404 0.001404      0.002483
##
## 2. Quantiles for each variable:
##
##      2.5%      25%      50%      75%      97.5%
## n  10.0000  10.0000  11.0000  13.0000  17.0000
## np  6.6301  8.7298  9.5128  10.0849  11.7442
## p   0.4823  0.7101  0.8259  0.9158  0.9917
```

```
library(coda)
```

```
# Low autocorrelation indicates convergence
autocorr.plot(samples)
```



```
autocorr(samples, lag = 1)
```

```
## [[1]]
## , , n
##
##          n          np          p
## Lag 1 0.4824136 -0.3968612 -0.6935977
##
## , , np
##
##          n          np          p
## Lag 1 0.07982781 -0.060955 -0.1100603
##
## , , p
##
##          n          np          p
## Lag 1 -0.3602373 0.2769471 0.5120752
##
##
## [[2]]
## , , n
##
##          n          np          p
## Lag 1 0.5007476 -0.398742 -0.7058926
##
## , , np
```

```
##
##           n           np           p
## Lag 1 0.09589331 -0.04156785 -0.1091882
##
## , , p
##
##           n           np           p
## Lag 1 -0.3570822 0.2951749 0.5183406
# high ESS indicates convergence
effectiveSize(samples)
```

```
##           n           np           p
## 3297.846 12665.398 3198.925
# R less than 1.1 indicates convergence
gelman.diag(samples)
```

```
## Potential scale reduction factors:
##
##      Point est. Upper C.I.
## n           1           1
## np          1           1
## p           1           1
##
## Multivariate psrf
##
## 1
# /z/ less than 2 indicates convergence
geweke.diag(samples[[1]])
```

```
##
## Fraction in 1st window = 0.1
## Fraction in 2nd window = 0.5
##
##           n           np           p
## -0.22358 -0.29722 -0.02711
```

We can see the model converges by the above results. n converges at about 12, p at 0.8 and np at 10.

```
Y<-10

data<-list(Y=Y)

library(rjags)

model_string<-textConnection("model{

  #Likelihood
  Y~dbin(p,n)

  #Priors
  lambda<-10
  a<-10
```

```

    b<-10
    n~dpois(lambda)
    p~dbeta(a,b)
    np<-n*p
  }")

#inits <- list(n=20, p=0.5)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

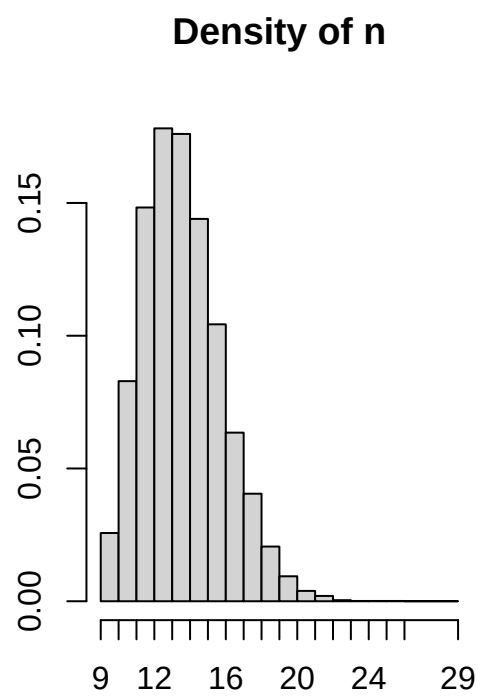
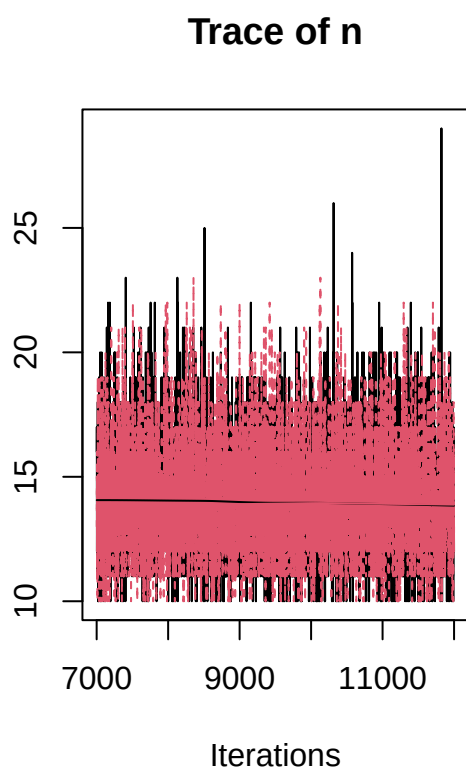
update(model, 2000, progress.bar = "none")

params <-c("n","p","np")

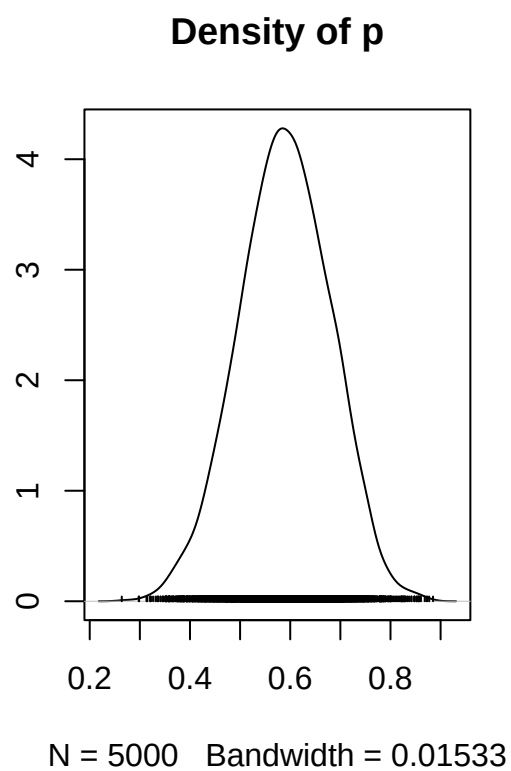
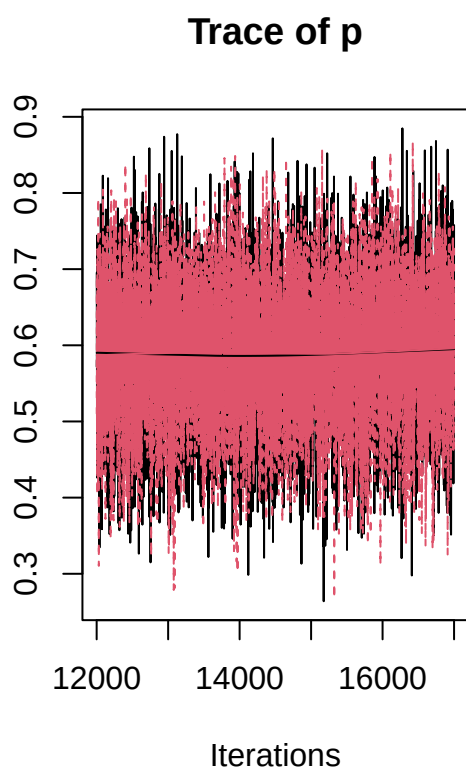
samples <- coda.samples(model,
                        variable.names = params,
                        n.iter = 5000, progress.bar = "none")
samples_n <- coda.samples(model,
                          variable.names = "n",
                          n.iter = 5000, progress.bar = "none")
samples_p <- coda.samples(model,
                          variable.names = "p",
                          n.iter = 5000, progress.bar = "none")
samples_np <- coda.samples(model,
                           variable.names = "np",
                           n.iter = 5000, progress.bar = "none")

plot(samples_n)

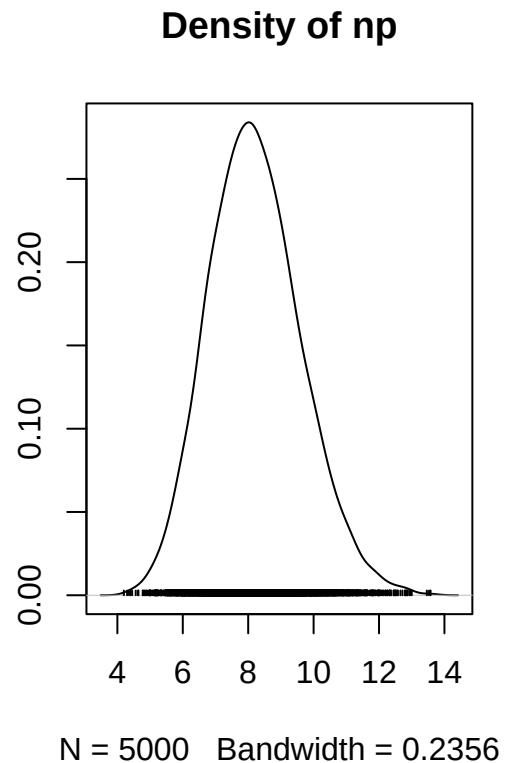
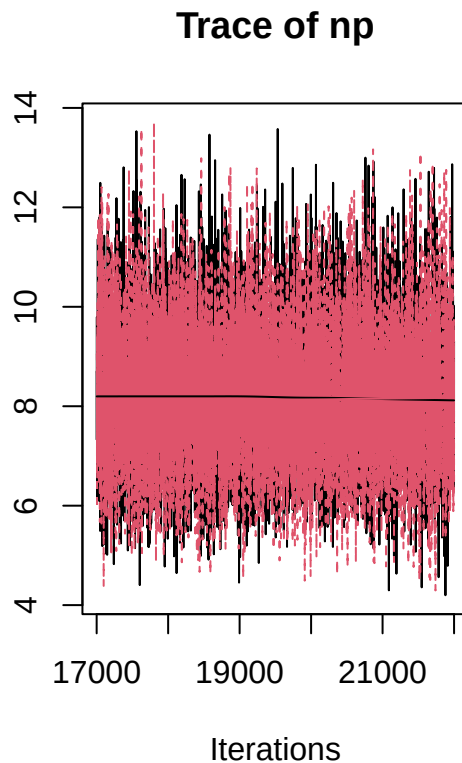
```



```
plot(samples_p)
```



```
plot(samples_np)
```

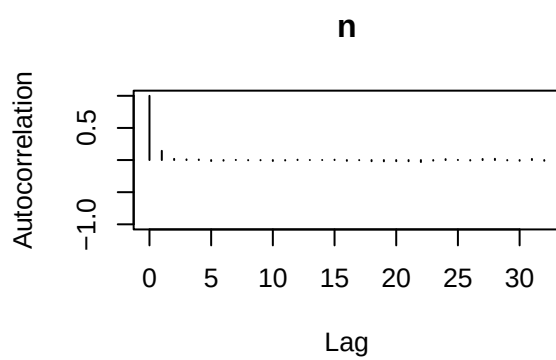
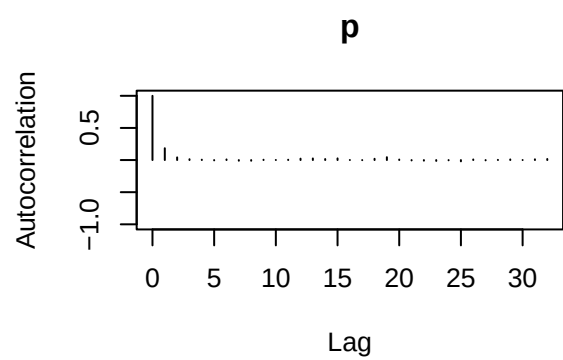
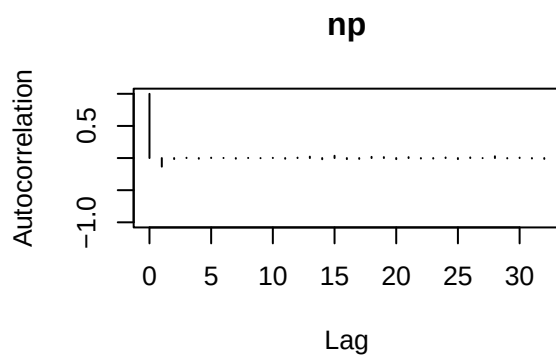
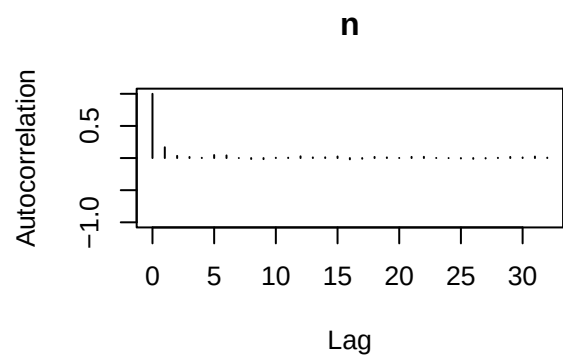


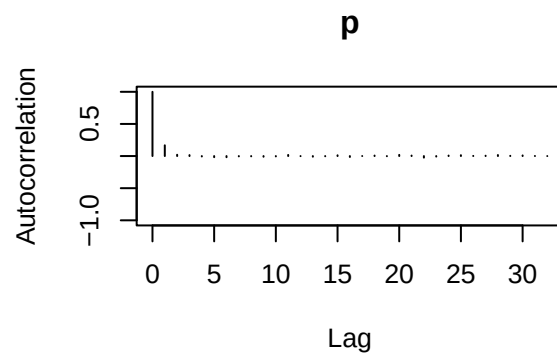
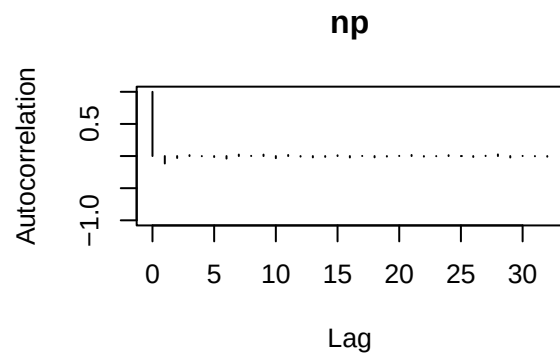
```
summary(samples)
```

```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##      Mean      SD Naive SE Time-series SE
## n  14.1528  2.24173 0.0224173      0.027573
## np  8.2471  1.39822 0.0139822      0.011633
## p   0.5887  0.09132 0.0009132      0.001092
##
## 2. Quantiles for each variable:
##
##      2.5%    25%    50%    75%   97.5%
## n  11.0000  12.0000  14.0000  16.0000  19.000
## np  5.6433  7.2987  8.1733  9.1527  11.195
## p   0.4086  0.5266  0.5903  0.6512  0.764
```

```
library(coda)
```

```
# Low autocorrelation indicates convergence
autocorr.plot(samples)
```





```
autocorr(samples, lag = 1)
```

```
## [[1]]
## , , n
##
##          n          np          p
## Lag 1 0.1682512 -0.2276944 -0.4151974
##
## , , np
##
##          n          np          p
## Lag 1 0.0700169 -0.1338857 -0.2173898
##
## , , p
##
##          n          np          p
## Lag 1 -0.08969961 0.0881749 0.1845692
##
##
## [[2]]
## , , n
##
##          n          np          p
## Lag 1 0.143854 -0.2281928 -0.3926219
##
## , , np
```

```
##
##           n           np           p
## Lag 1 0.07786893 -0.1161716 -0.2045352
##
## , , p
##
##           n           np           p
## Lag 1 -0.0598263 0.09863226 0.1678592
# high ESS indicates convergence
effectiveSize(samples)
```

```
##           n           np           p
## 6715.159 14461.201 7003.163
# R less than 1.1 indicates convergence
gelman.diag(samples)
```

```
## Potential scale reduction factors:
##
##      Point est. Upper C.I.
## n           1           1
## np          1           1
## p           1           1
##
## Multivariate psrf
##
## 1
# /z/ less than 2 indicates convergence
geweke.diag(samples[[1]])
```

```
##
## Fraction in 1st window = 0.1
## Fraction in 2nd window = 0.5
##
##           n           np           p
## -0.1697 -2.0727 -1.3827
```

We can see the model converges by the above results. n converges at about 14, p at 0.6 and np at 8.

Question 4

We set up a two sample t-test to test whether the treatment is effective or not. We take $Y_i \sim \text{Normal}(\mu, \sigma^2)$ for placebo observations and $Y_i \sim \text{Normal}(\mu + \delta, \sigma^2)$ for treatment observations. We test Whether $\delta = 0$ or not.

```
n<-12
n1<-6
n2<-6
Y1<-c(2.0,-3.1,-1.0,0.2,0.3,0.4)
Y2<-c(-3.5,-1.6,-4.6,-0.9,-5.1,0.1)

Y1_m<-mean(Y1)
Y2_m<-mean(Y2)

sigma1<-mean((Y1_m-Y1)^2)
```

```

sigma2<-mean((Y2_m-Y2)^2)
sig_rt<-sqrt(sigma1/2+sigma2/2)

p_mean<-Y2_m-Y1_m
sigma_hat<-(sig_rt)*sqrt(1/n1+1/n2)

cred_set <- p_mean+sigma_hat*qt(c(0.025,0.975),df=n1+n2)

p_mean;sigma_hat;cred_set

```

```

## [1] -2.4
## [1] 1.012423
## [1] -4.6058799 -0.1941201

```

The 95% credible set doesn't include 0. So, we can say that the treatment is effective. To test for sensitivity to the prior we also fit the model using proper priors using JAGS.

```

library(rjags)
data <- list(n=6,Y1=Y1,Y2=Y2)

model_string <- textConnection("model{

  # Likelihood
  for(i in 1:n){
    Y1[i] ~ dnorm(mu,tau)
    Y2[i] ~ dnorm(mu+delta,tau)
  }

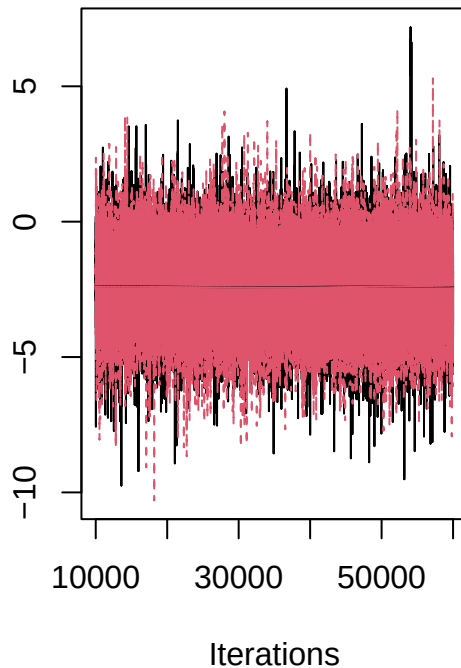
  # Priors
  mu ~ dnorm(0, 0.0001)
  delta ~ dnorm(0, 0.0001)
  tau ~ dgamma(0.1, 0.1)
  sigma <- 1/sqrt(tau)
}")

model <- jags.model(model_string,data = data, n.chains=2,quiet=TRUE)
update(model, 10000, progress.bar="none")
params <- c("delta")
samples <- coda.samples(model,
                        variable.names=params,
                        n.iter=50000, progress.bar="none")

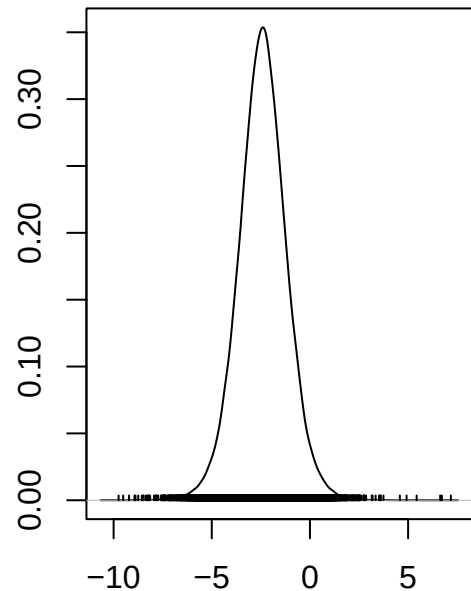
plot(samples)

```

Trace of delta



Density of delta



N = 50000 Bandwidth = 0.1223

```
summary(samples)
```

```
##
## Iterations = 10001:60000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 50000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean           SD      Naive SE Time-series SE
##      -2.410054      1.232668      0.003898      0.006846
##
## 2. Quantiles for each variable:
##
##      2.5%      25%      50%      75%      97.5%
## -4.86818 -3.18148 -2.41124 -1.63602  0.04383
```

We can see that the 95% credible interval now includes 0. So, we can say that the conclusion is slightly sensitive to prior.

Question 5

```
library(MASS)
```

```
## Warning: package 'MASS' was built under R version 4.2.3
```

```

# Load the Boston dataset from the MASS library
data(Boston)
n <- nrow(Boston)
p <- ncol(Boston)
X <- cbind(1, Boston[, -p])

# Load necessary libraries
library(rjags)
library(coda)

# Define the data for the model
data_list <- list(Y = Boston$medv, n = n, p = p, X = X)

# Define the JAGS model for part a
model_string_a <- "
model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(inprod(X[i, ], beta[1:p]), tau)
  }

  # Priors
  beta[1] ~ dnorm(0,1e-4)
  for(i in 2:p){
    beta[i] ~ dnorm(0, 1e-4)
  }
  tau ~ dgamma(0.1, 0.1)
  sigma2 <- 1 / tau
}"

# Compile the JAGS model
model_a <- jags.model(textConnection(model_string_a), data = data_list, quiet = TRUE)

# Burn-in
update(model_a, 5e3, progress.bar = 'none')

# Sampling from the posterior distribution
params <- c('beta')
samples_a <- coda.samples(model_a,
                          variable.names = params,
                          n.iter = 2e4, progress.bar = 'none')

# Summarize the posterior samples
summary(samples_a)

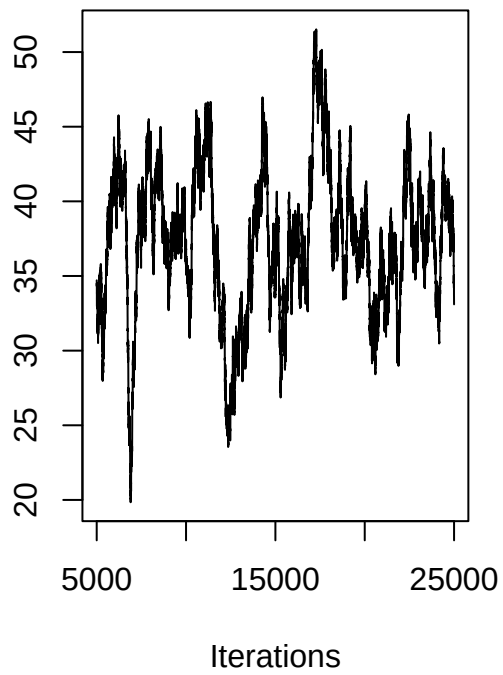
##
## Iterations = 5001:25000
## Thinning interval = 1
## Number of chains = 1
## Sample size per chain = 20000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##

```

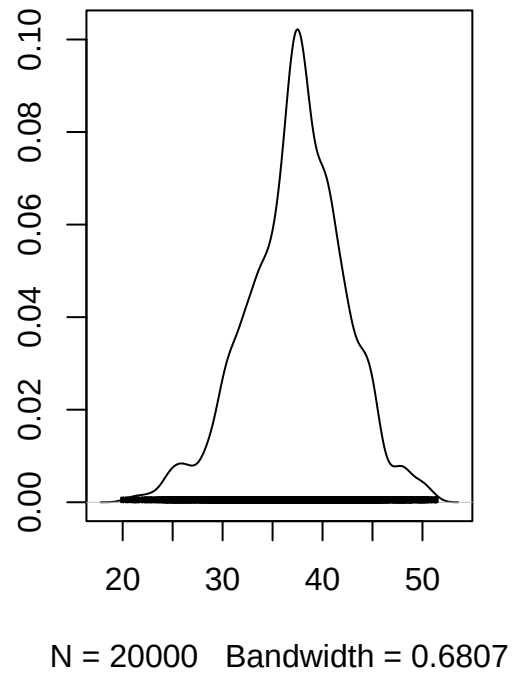
```
##           Mean           SD Naive SE Time-series SE
## beta[1]  37.361234  4.942992  3.495e-02    1.1761678
## beta[2]  -0.108636  0.032748  2.316e-04    0.0005036
## beta[3]   0.045548  0.013606  9.621e-05    0.0004390
## beta[4]   0.019747  0.061768  4.368e-04    0.0026658
## beta[5]   2.672453  0.859388  6.077e-03    0.0119366
## beta[6]  -18.018841  3.834759  2.712e-02    0.3762833
## beta[7]   3.766922  0.395368  2.796e-03    0.0466201
## beta[8]   0.001098  0.014063  9.944e-05    0.0007160
## beta[9]  -1.475349  0.196012  1.386e-03    0.0110603
## beta[10]  0.310120  0.065276  4.616e-04    0.0037121
## beta[11] -0.012318  0.003694  2.612e-05    0.0002766
## beta[12] -0.980058  0.129718  9.172e-04    0.0145735
## beta[13]  0.009249  0.002707  1.914e-05    0.0001349
## beta[14] -0.526071  0.049376  3.491e-04    0.0022715
##
## 2. Quantiles for each variable:
##
##           2.5%           25%           50%           75%           97.5%
## beta[1]  26.345456  34.309680  37.554800  40.54663  47.055016
## beta[2]  -0.172230  -0.130611  -0.108741  -0.08660  -0.045046
## beta[3]   0.018511  0.036393  0.045523  0.05467  0.072432
## beta[4]  -0.101663  -0.021928  0.020138  0.06182  0.139497
## beta[5]   0.991706  2.096344  2.670721  3.25100  4.346946
## beta[6] -25.468674 -20.657294 -17.884725 -15.47386 -10.318348
## beta[7]   3.044275  3.484212  3.763642  4.03842  4.541623
## beta[8]  -0.027334  -0.008256  0.001409  0.01055  0.028422
## beta[9]  -1.860986  -1.607461  -1.475886  -1.34230  -1.086832
## beta[10]  0.180411  0.266781  0.310320  0.35357  0.437212
## beta[11] -0.019247  -0.014915  -0.012355  -0.00974  -0.005107
## beta[12] -1.223128  -1.071458  -0.987154  -0.88736  -0.727127
## beta[13]  0.003927  0.007407  0.009246  0.01110  0.014510
## beta[14] -0.621775  -0.560077  -0.526208  -0.49194  -0.430333
```

```
# Plot trace and density for beta[1] and beta[2]
par(mfrow=c(2,1))
plot(samples_a[[1]][, "beta[1]"], type="l", main="Trace Plot for beta[1]")
```

Trace Plot for beta[1]

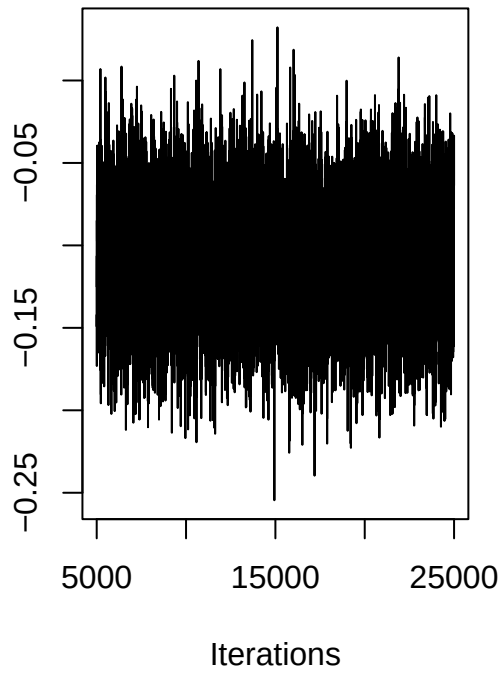


Trace Plot for beta[1]

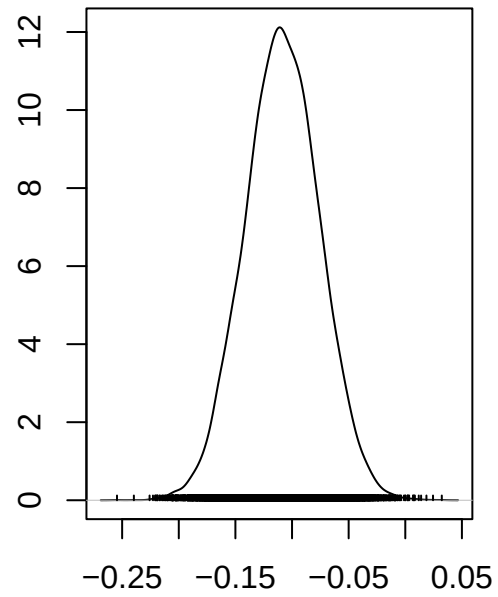


```
plot(samples_a[[1]][, "beta[2]"], type="l", main="Trace Plot for beta[2]")
```

Trace Plot for beta[2]



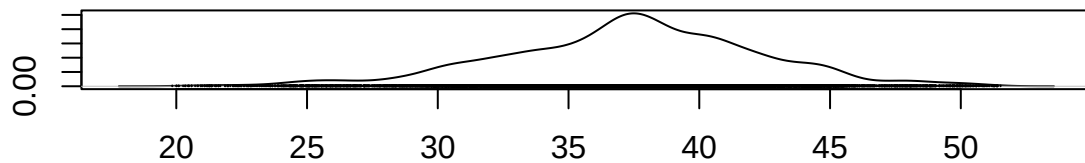
Trace Plot for beta[2]



N = 20000 Bandwidth = 0.004789

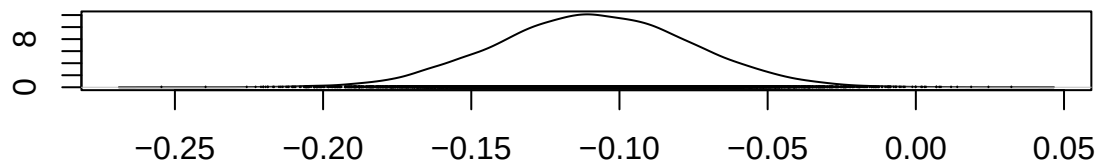
```
par(mfrow=c(2,1))
densplot(samples_a[[1]][, "beta[1]"], main="Density Plot for beta[1]")
densplot(samples_a[[1]][, "beta[2]"], main="Density Plot for beta[2]")
```


Density Plot for beta[1]



N = 20000 Bandwidth = 0.6807

Density Plot for beta[2]



N = 20000 Bandwidth = 0.004789

```
# Part b - Fit linear regression using lm
fit_least_sq <- lm(medv ~ ., data = Boston)
summary(fit_least_sq)
```

```
##
## Call:
## lm(formula = medv ~ ., data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.595  -2.730  -0.518   1.777   26.199
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.646e+01  5.103e+00   7.144 3.28e-12 ***
## crim        -1.080e-01  3.286e-02  -3.287 0.001087 **
## zn           4.642e-02  1.373e-02   3.382 0.000778 ***
## indus        2.056e-02  6.150e-02   0.334 0.738288
## chas         2.687e+00  8.616e-01   3.118 0.001925 **
## nox          -1.777e+01  3.820e+00  -4.651 4.25e-06 ***
## rm           3.810e+00  4.179e-01   9.116 < 2e-16 ***
## age          6.922e-04  1.321e-02   0.052 0.958229
## dis          -1.476e+00  1.995e-01  -7.398 6.01e-13 ***
## rad           3.060e-01  6.635e-02   4.613 5.07e-06 ***
## tax          -1.233e-02  3.760e-03  -3.280 0.001112 **
## ptratio      -9.527e-01  1.308e-01  -7.283 1.31e-12 ***
```

```
## black          9.312e-03  2.686e-03   3.467 0.000573 ***
## lstat          -5.248e-01  5.072e-02 -10.347 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.745 on 492 degrees of freedom
## Multiple R-squared:  0.7406, Adjusted R-squared:  0.7338
## F-statistic: 108.1 on 13 and 492 DF,  p-value: < 2.2e-16

# Part c - Define the JAGS model for part c
model_string_c <- "
model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(inprod(X[i, ], beta[]), tau)
  }

  # Priors
  beta[1] ~ ddexp(0,1e-4)
  for(i in 2:p){
    beta[i] ~ ddexp(0, 1e-4)
  }
  tau ~ dgamma(0.1, 0.1)
  sigma2 <- 1 / tau
}"

# Compile the JAGS model for part c
model_c <- jags.model(textConnection(model_string_c), data = data_list, quiet = TRUE)

# Burn-in
update(model_c, 5e3, progress.bar = 'none')

# Sampling from the posterior distribution for part c
samples_c <- coda.samples(model_c,
                          variable.names = params,
                          n.iter = 2e4, progress.bar = 'none')

# Summarize the posterior samples for part c
summary(samples_c)

##
## Iterations = 6001:26000
## Thinning interval = 1
## Number of chains = 1
## Sample size per chain = 20000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean          SD Naive SE Time-series SE
## beta[1]  36.721701 4.439323 3.139e-02    1.0668106
## beta[2]  -0.107492 0.033245 2.351e-04    0.0006190
## beta[3]   0.045943 0.013740 9.716e-05    0.0005343
## beta[4]   0.023437 0.060113 4.251e-04    0.0030047
## beta[5]   2.693862 0.861921 6.095e-03    0.0119682
```

```

## beta[6] -18.161909 3.669023 2.594e-02 0.4104853
## beta[7] 3.803096 0.398054 2.815e-03 0.0547055
## beta[8] 0.001686 0.012840 9.079e-05 0.0006748
## beta[9] -1.469772 0.184682 1.306e-03 0.0101478
## beta[10] 0.307772 0.065352 4.621e-04 0.0040543
## beta[11] -0.012226 0.003597 2.543e-05 0.0003068
## beta[12] -0.967468 0.135284 9.566e-04 0.0180134
## beta[13] 0.009575 0.002547 1.801e-05 0.0001278
## beta[14] -0.525005 0.050355 3.561e-04 0.0026891
##
## 2. Quantiles for each variable:
##
##          2.5%      25%      50%      75%      97.5%
## beta[1] 27.329936 34.163787 36.632678 39.443453 45.661549
## beta[2] -0.173174 -0.129912 -0.107465 -0.085011 -0.042873
## beta[3] 0.019064 0.036570 0.045859 0.055301 0.073229
## beta[4] -0.090313 -0.017819 0.021313 0.064806 0.141936
## beta[5] 1.006044 2.111271 2.691519 3.272934 4.371106
## beta[6] -25.355364 -20.620364 -18.274786 -15.674020 -10.790982
## beta[7] 2.995148 3.533354 3.808458 4.062776 4.575563
## beta[8] -0.023827 -0.006744 0.001664 0.010115 0.027383
## beta[9] -1.841404 -1.591448 -1.467852 -1.346057 -1.115363
## beta[10] 0.189030 0.261149 0.304336 0.352567 0.440000
## beta[11] -0.019497 -0.014649 -0.012077 -0.009665 -0.005622
## beta[12] -1.226794 -1.057736 -0.971657 -0.874933 -0.701405
## beta[13] 0.004494 0.007891 0.009601 0.011288 0.014504
## beta[14] -0.623752 -0.559950 -0.524826 -0.490398 -0.425838

```